

# 파이토치 딥러닝 프로그래밍

## 20. Transformer\_ViT

▶ Transformer & ViT – 최신 딥러닝 모델 통합 이해

- Self-Attention(Q/K/V)와 Multi-Head Attention의 수학적 원리 이해
- Transformer Encoder 구조와 Positional Encoding 메커니즘 파악
- Vision Transformer(ViT) 이미지 분류 파인튜닝 실습
- LSTM vs Transformer, CNN vs ViT 성능 비교 분석

### 순차 처리의 근본적 한계

RNN과 LSTM은 시퀀스 데이터를 시간 순서대로 처리해야 하므로 병렬 연산이 불가능합니다.  $t$  시점의 hidden state는  $t-1$  시점의 계산이 완료되어야만 구할 수 있어 GPU의 대규모 병렬 처리 능력을 활용할 수 없습니다.

### 장거리 의존성 문제

LSTM이 gradient vanishing을 완화했지만, 여전히 긴 시퀀스에서 초반 정보가 희석되는 문제가 있습니다. 100개 이상의 토큰을 처리할 때 문맥 정보 손실이 발생하며, 이는 기계번역과 문서 이해 태스크에서 치명적입니다.

### 계산 효율성 저하

배치 내 시퀀스 길이가 다를 경우 패딩으로 인한 연산 낭비가 발생합니다. 또한 recurrent 구조 특성상 메모리 사용량이 시퀀스 길이에 비례하여 증가하며, 긴 문장 처리 시 out-of-memory 에러가 빈번합니다.

## 직관적 이해

Self-Attention은 각 단어가 문장 내 모든 단어와의 관련성을 동시에 계산합니다. 이는 데이터베이스 검색과 유사한 메커니즘입니다.

- Query(Q): "내가 찾고자 하는 정보는 무엇인가?" - 현재 단어의 질문
- Key(K): "나는 어떤 정보를 가지고 있는가?" - 각 단어의 인덱스
- Value(V): "실제로 전달할 정보" - 각 단어의 실제 표현

각 단어의 Query가 모든 단어의 Key와 내적하여 유사도(attention score)를 계산하고, 이를 가중치로 Value를 가중합하여 최종 출력을 생성합니다.

## Scaled Dot-Product Attention 수식

이 메커니즘은 모든 위치 간 관계를 동시에 계산하므로  $O(n^2)$  복잡도를 가지지만, 완벽한 병렬 처리가 가능하여 GPU에서 매우 빠릅니다.

### Step 1: Attention Score 계산

Query와 Key의 내적으로 유사도 계산

$$\text{Score} = QK^T$$

행렬 차원:  $(\text{seq\_len}, d_k) \times (d_k, \text{seq\_len})$   
 $= (\text{seq\_len}, \text{seq\_len})$

### Step 2: Scaling

차원의 제곱근으로 나누어 gradient 안정화

$$\text{Scaled Score} = \frac{QK^T}{\sqrt{d_k}}$$

$d_k$ 가 크면 내적 값이 커져 softmax가 극단적 확률을 출력하므로 스케일링 필수

### Step 3: Softmax & Weighted Sum

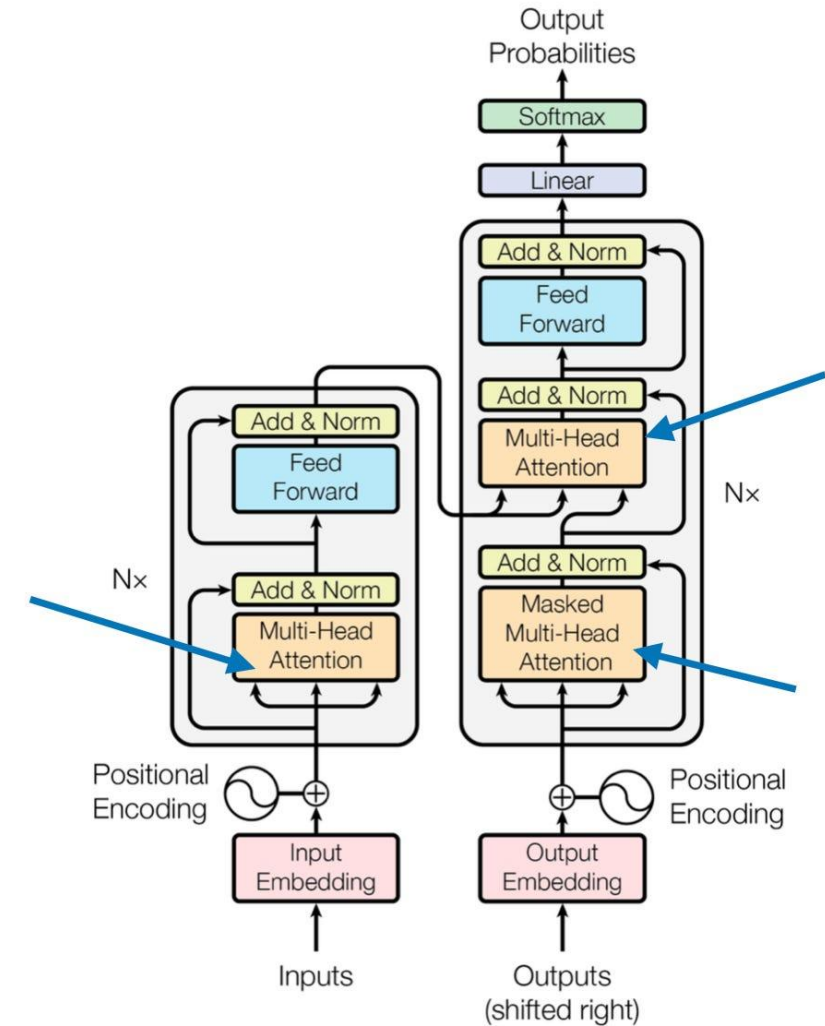
확률 분포로 변환 후 Value와 가중합

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

최종 출력 차원:  $(\text{seq\_len}, d_v)$

Multi-Head Attention은 입력 Query, Key, Value를 각각 여러 개의 "head"로 분할하고, 각 head마다 독립적으로 attention을 계산한 후 결과를 결합하는 방식으로 작동합니다.

이는 모델이 여러 표현 공간에서 동시에 정보를 집중할 수 있도록 합니다.



1

**선형 변환 및 분할**

입력  $Q, K, V$ 를 각각  $h$ 개의 다른 선형 변환( $W_Q, W_K, W_V$  행렬)을 통해 저차원 공간으로 투영하고, 각 head에 맞게 분할합니다.

$$Q_i = QW_{Q_i}, \quad K_i = KW_{K_i}, \quad V_i = VW_{V_i}$$

여기서  $W_{Q_i}, W_{K_i}, W_{V_i}$ 는  $i$ 번째 head의 가중치 행렬입니다.

2

**각 Head별 Scaled Dot-Product Attention 계산**

분할된  $Q_i, K_i, V_i$ 를 사용하여 각 head에서 독립적으로 Scaled Dot-Product Attention을 수행합니다.

$$\text{Attention}_i(Q, K, V) = \text{softmax} \left( \frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i$$

3

**결과 Concatenation 및 최종 투영**

각 head의 attention 결과를 모두 연결(concatenate)한 다음, 하나의 최종 선형 변환( $W_O$  행렬)을 적용하여 원래 차원으로 복원합니다.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

여기서  $\text{head}_i = \text{Attention}_i(Q, K, V)$  입니다.

각 attention head는 서로 다른 가중치 행렬을 학습하므로, 문장 내에서 다양한 유형의 관계에 집중할 수 있습니다. 이는 단일 attention으로는 포착하기 어려운 복합적인 문맥 이해를 가능하게 합니다.



### 구문적 관계

한 head는 주어-동사, 수식어-피수식어 등 문법적 구조를 파악하는 데 집중할 수 있습니다.



### 의미론적 유사성

다른 head는 동의어나 유사한 의미를 가진 단어들을 연결하여 의미적 문맥을 형성할 수 있습니다.



### 대명사 및 개체명 참조

"그녀", "그것"과 같은 대명사가 가리키는 실제 개체나 명명된 개체(예: 회사, 사람 이름) 간의 관계를 추적할 수 있습니다.



### 장거리 의존성

문장의 앞부분과 뒷부분에 있는 단어들 사이의 숨겨진 관계나 맥락을 이해하는 데 특화될 수 있습니다.





### 단일 Head Attention의 한계

- 하나의 관점만 학습하여 복잡한 언어적 관계를 완전히 포착하기 어려움.
- 제한된 표현력으로 인해 모델 성능 향상에 한계가 있음.

### Multi-Head Attention의 장점

- 여러 개의 head가 병렬로 다양한 특징을 학습하여 모델의 표현력과 이해도 대폭 향상.
- 각 head가 독립적으로 학습하므로 다양한 문맥적 정보를 동시에 처리하여 더 견고하고 정확한 예측 가능.
- 특히 긴 문장에서 장거리 의존성 포착 능력 우수.

Google의 BERT와 같은 최신 트랜스포머 모델은 Multi-Head Attention을 적극적으로 활용합니다.

예를 들어, BERT-base 모델은 일반적으로 12개의 attention head를 사용하며, 각 head의 차원은 64 ( $d_{\text{model}}/h = 768/12$ )로 설정됩니다.

이는 모델이 매우 다양한 언어적 패턴을 동시에 학습하고 통합할 수 있도록 하여 뛰어난 성능을 달성하는 핵심 요인 중 하나입니다.

### → Self-Attention의 맹점

Attention 메커니즘은 단어의 순서 정보를 고려하지 않습니다. "개가 고양이를 쫓았다"와 "고양이가 개를 쫓았다"가 같은 attention score를 가질 수 있습니다.

### → 사인/코사인 함수 활용

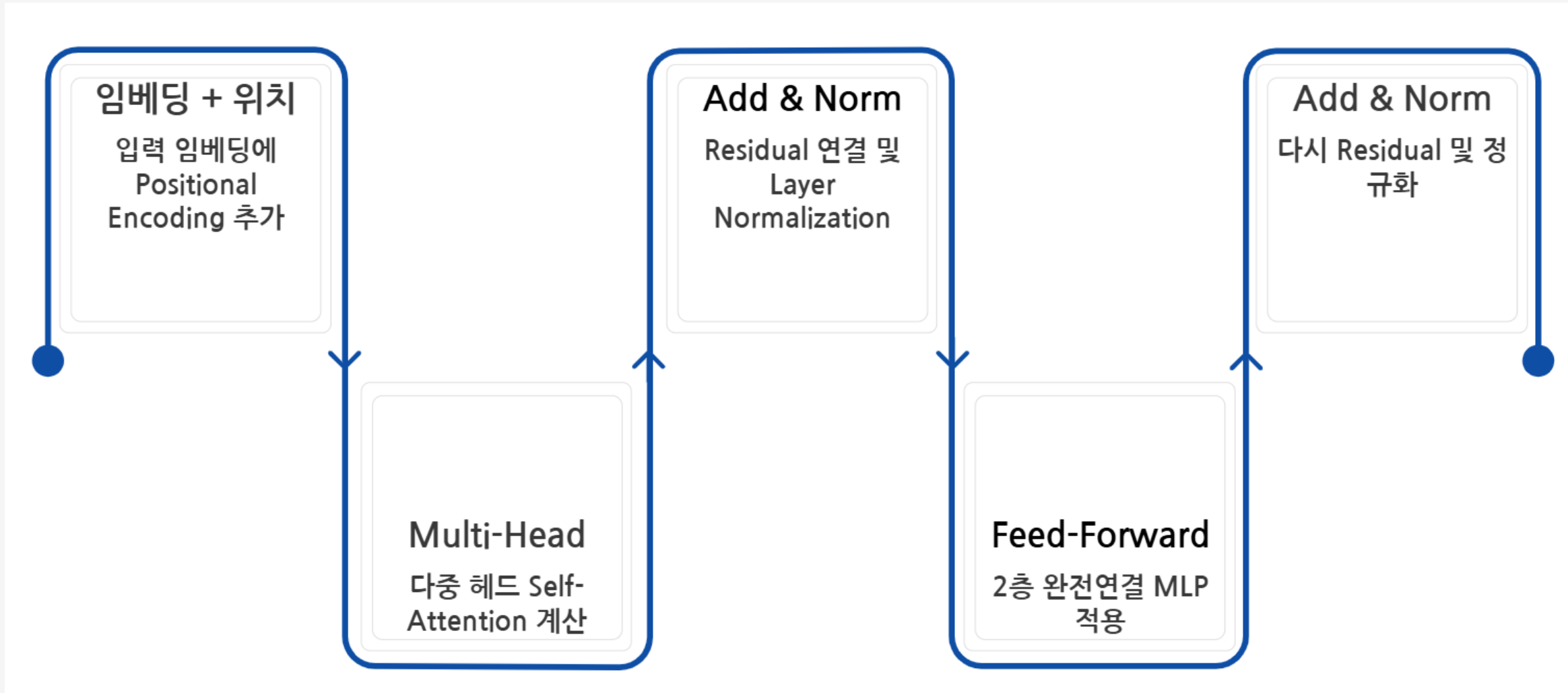
각 위치 pos와 차원 i에 대해 고정된 사인·코사인 값을 더해 위치 정보를 주입합니다.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

### → 학습 불필요한 고정 패턴

사인·코사인 함수는 학습 없이 상대적 위치 관계를 표현할 수 있어 임의의 길이 시퀀스에 일반화됩니다. 최근 모델(BERT, GPT)은 학습 가능한 positional embedding도 사용합니다.



## 핵심 구성 요소

1. Multi-Head Self-Attention: 입력 시퀀스 내 모든 위치 간 관계 모델링
2. Residual Connection: 입력을 출력에 더하여 gradient flow 개선
3. Layer Normalization: 각 층의 출력을 정규화하여 학습 안정화
4. Feed-Forward Network: 위치별로 독립적인 2층 MLP 적용  
( $d_{\text{model}} \rightarrow 4 \times d_{\text{model}} \rightarrow d_{\text{model}}$ )

## 설계 철학

- Residual connection과 Layer Normalization은 **깊은 네트워크 학습을 가능하게** 합니다. BERT는 12~24층, GPT-3는 96층의 Encoder/Decoder를 쌓아 고수준 표현을 학습합니다.
- FFN은 attention으로 모은 정보를 **비선형 변환**하여 복잡한 패턴을 포착합니다. ReLU나 GELU 활성화 함수를 사용합니다.



## 사전학습 (Pre-training)

대규모 텍스트 코퍼스(Wikipedia, BookCorpus)에서 Masked Language Model(MLM)과 Next Sentence Prediction(NSP) 태스크로 학습합니다. MLM은 문장의 15% 토큰을 마스킹하고 원래 단어를 예측하도록 훈련합니다.



## 파인튜닝 (Fine-tuning)

사전학습된 BERT에 태스크별 분류 헤드를 추가하고 소량의 라벨 데이터로 fine-tuning합니다. 감정 분석, 개체명 인식, 질의응답 등 다양한 downstream task에 적용 가능합니다.

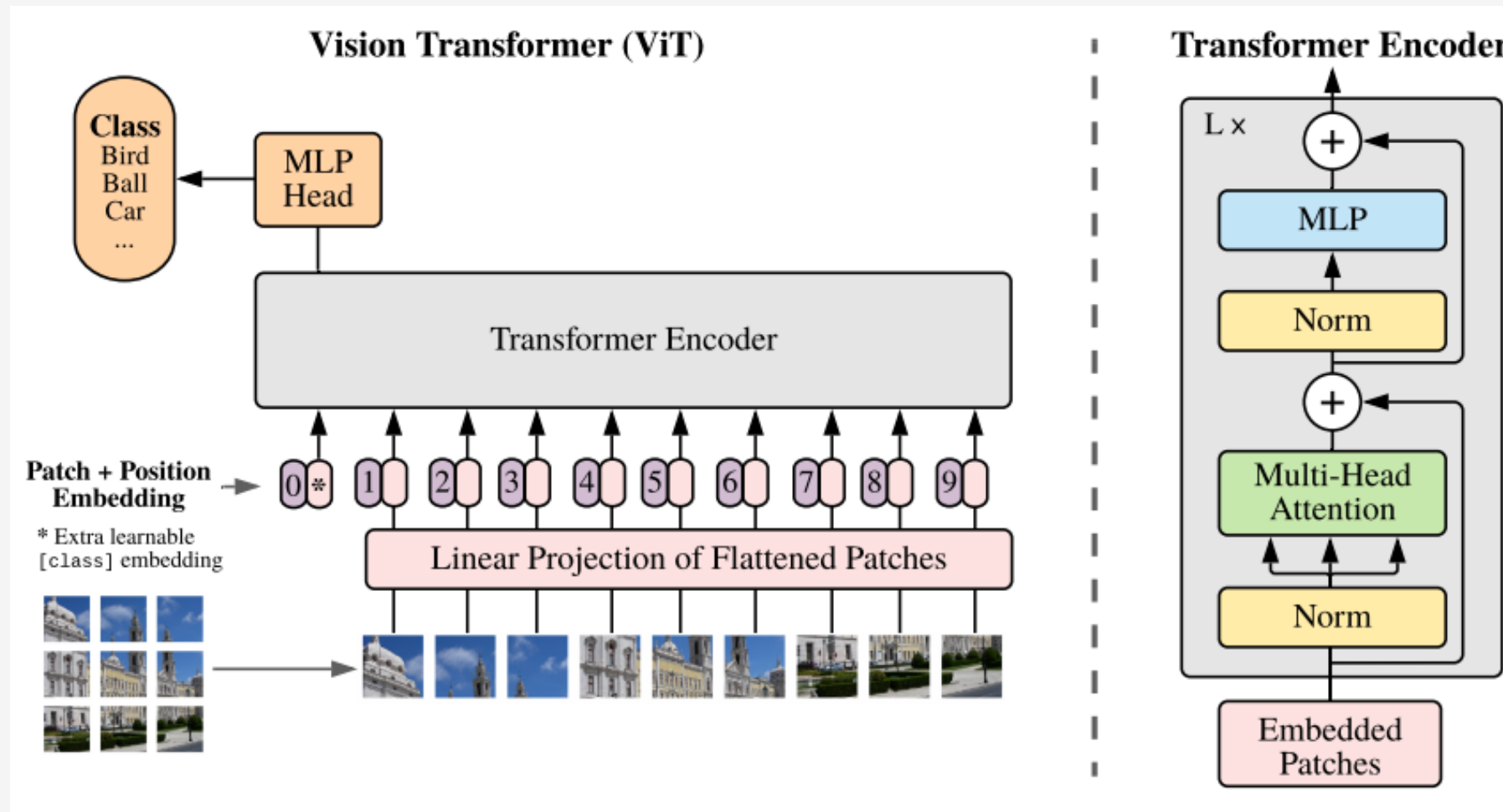


## DistilBERT의 효율성

BERT의 지식을 증류(distillation)하여 파라미터를 40% 줄이고 속도를 60% 향상시킨 경량화 모델입니다. 성능은 BERT의 97% 수준을 유지하며 실시간 서비스에 적합합니다.

## Vision Transformer (ViT): 이미지를 시퀀스로

ViT는 **CNN의 inductive bias(locality, translation equivariance) 없이** 순수 attention으로 이미지를 처리합니다. 대규모 데이터(ImageNet-21K 이상)에서 사전학습하면 CNN을 능가하는 성능을 보입니다.





## 이미지 패치 분할

224×224 이미지를 16×16 크기의 패치로 나누면  $14 \times 14 = 196$ 개의 패치가 생성됩니다. 각 패치는  $16 \times 16 \times 3 = 768$ 차원 벡터로 flatten됩니다.

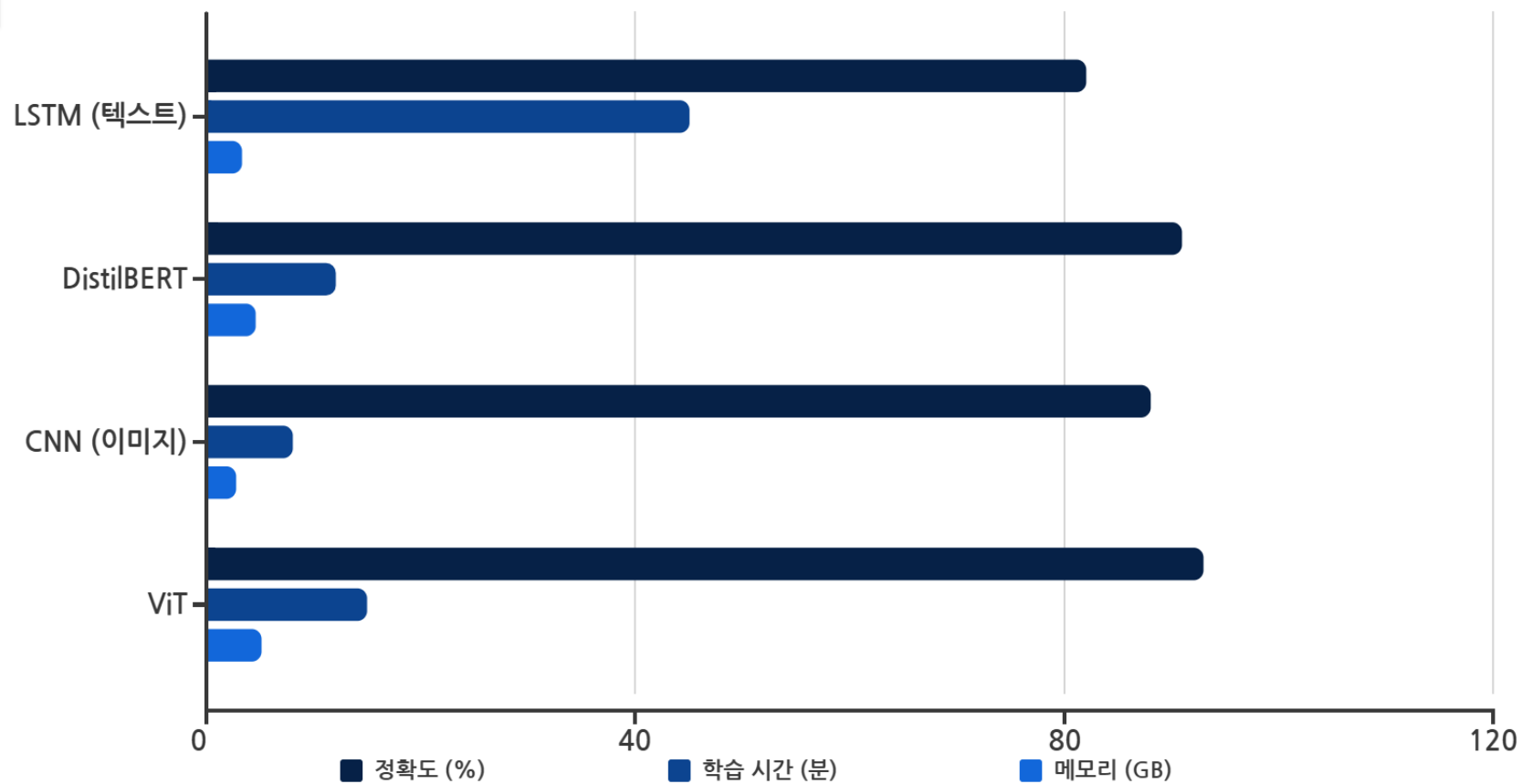
## 선형 투영 및 위치 임베딩

각 패치를 선형 변환하여  $d_{\text{model}}$  차원(768)으로 매핑하고, 학습 가능한 positional embedding을 더합니다. 추가로 CLS 토큰을 앞에 붙여 분류에 사용합니다.

## Transformer Encoder 적용

BERT와 동일한 Transformer Encoder를 통과시킵니다. ViT-Base는 12층, 12 heads, 768 hidden dim을 사용하며, 최종 CLS 토큰의 표현을 분류 헤드에 입력합니다.

# 모델 성능비교 : LSTM vs Transformer / CNN vs ViT



## 정확도

Transformer 계열이 9~11%p 높은 정확도를 기록했습니다. Self-Attention의 전역적 문맥 파악 능력이 핵심 요인입니다.

## 학습 시간

DistilBERT는 LSTM 대비 3.7배 빠르며, 병렬 처리 최적화 덕분입니다. ViT는 CNN보다 약간 느리지만 대규모 데이터에서는 역전됩니다.

## 메모리 사용량

Transformer는 attention matrix( $O(n^2)$ )로 인해 메모리가 많이 필요합니다. 긴 시퀀스 처리 시 gradient checkpointing 등 최적화 필수입니다.